// trabalho 4 · arquitetura mvvm

DuoCode

Reorganização em MVVM, com Repository, StateFlow e ViewModelFactory

André Caires · Gustavo Cáceres · Brenno Soares

// como estava e o que mudou

O que mudou do Trabalho 3 para o 4

trabalho 3 · a tela fazia tudo

- `AppNavigation` criava o banco e guardava as listas em `mutableStateListOf`
- Cada `@Composable` chamava o DAO em `CoroutineScope(Dispatchers.IO)`
- `recarregarListas(...)` depois de cada gravação
- Estado espalhado em vários `remember`
- Dados de exemplo num `LaunchedEffect`

→

trabalho 4 · cada camada com sua função

- Cada tela recebe o seu ViewModel com `viewModel(factory = ...)`
- Tela → ViewModel → Repository → DAO → Room
- `buscarTodos()` retorna `Flow`, e a tela atualiza sozinha
- Estado da tela num `UiState`, exposto por `StateFlow`
- Dados de exemplo no `Callback.onOpen` do `AppDatabase`

O app continua fazendo a mesma coisa: a mudança é na organização do código.

Por que mudou

saiu

DAO e corrotinas dentro da tela

O que era: `dao`, `CoroutineScope` e `recarregarListas` nos `@Composable`.

Ganho: a tela só desenha o estado e avisa o ViewModel quando o usuário clica.

evoluiu

DAOs, telas e dados iniciais

O que mudou: `buscarTodos()` virou `Flow` e deixou de ser `suspend`. O seed foi para o `AppDatabase`.

Por quê: o Room emite a lista de novo sempre que a tabela muda.

nova

Repository, ViewModel, UiState e Factory

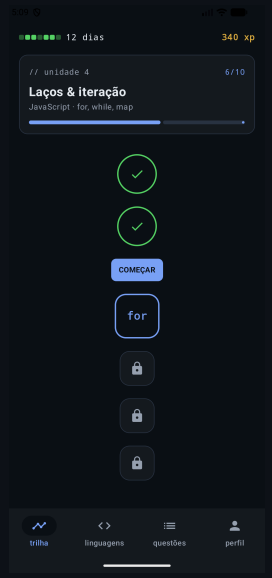
Por quê: só o Repository fala com o DAO e só o ViewModel fala com o Repository.

Ganho: se as questões vierem de uma API, só o Repository muda.

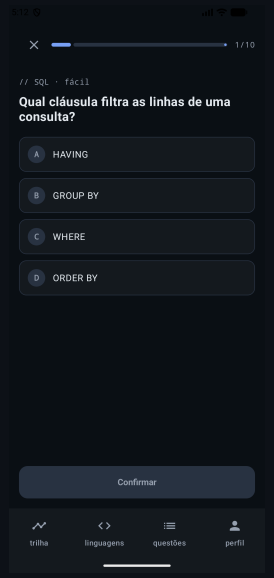
As 8 telas

fluxo de estudo

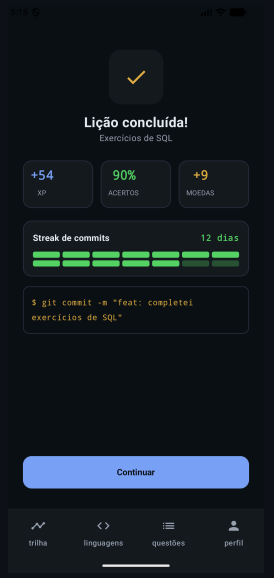
listas + detalhes



Trilha



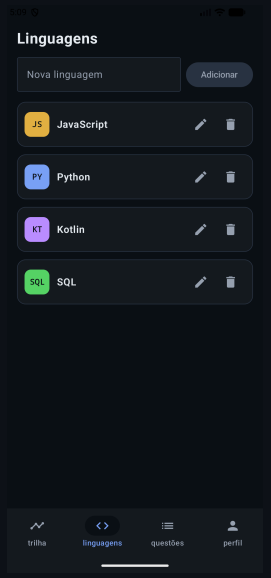
Quiz



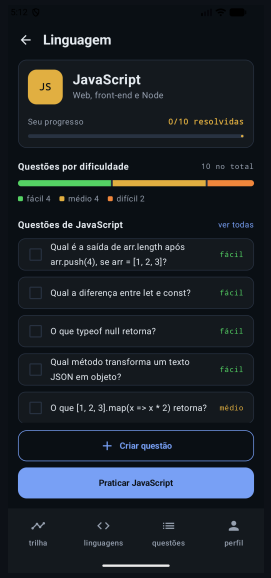
Resultado



Perfil



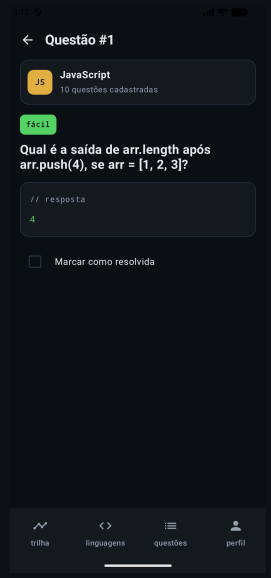
Linguagens



Det. ling.



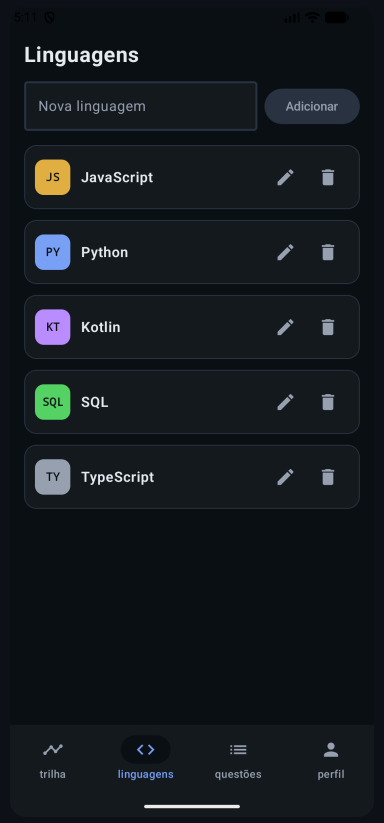
Questões



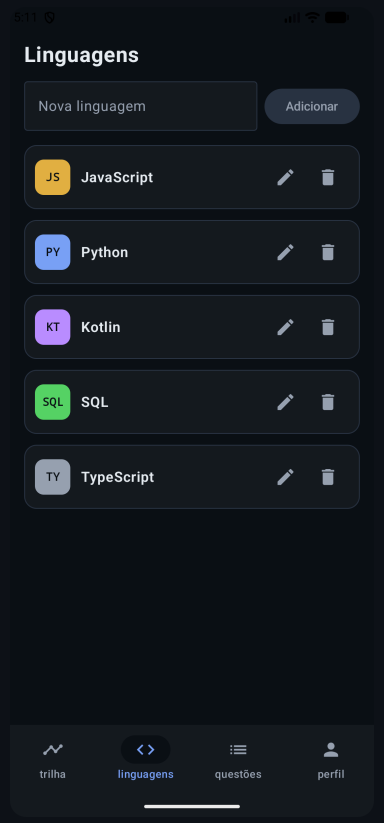
Det. quest.

// app rodando no emulador

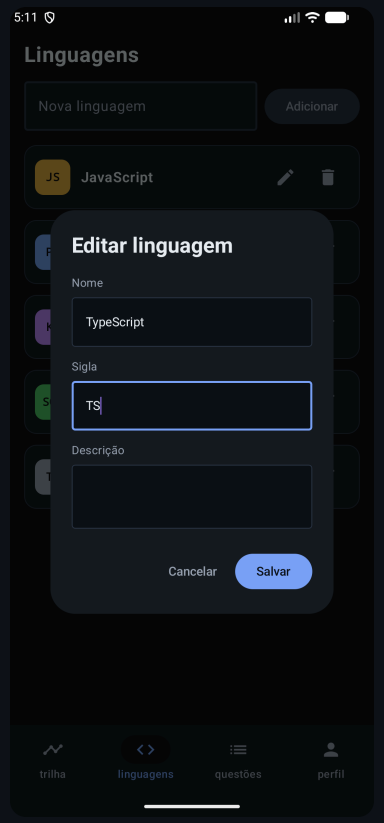
Lista de Linguagens no banco



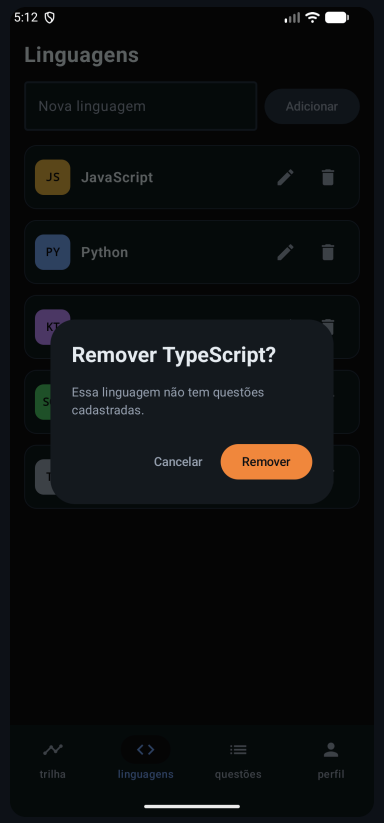
adicionar



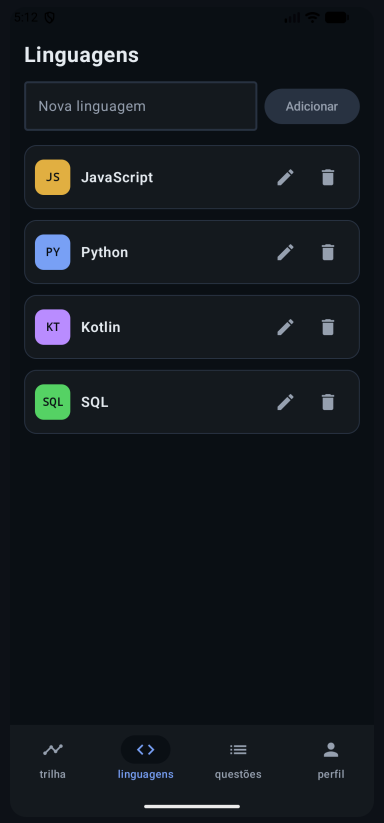
fechar e abrir



editar



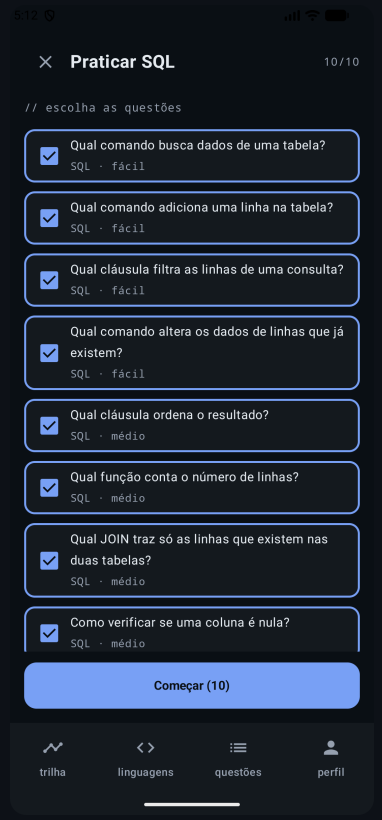
confirmar remoção



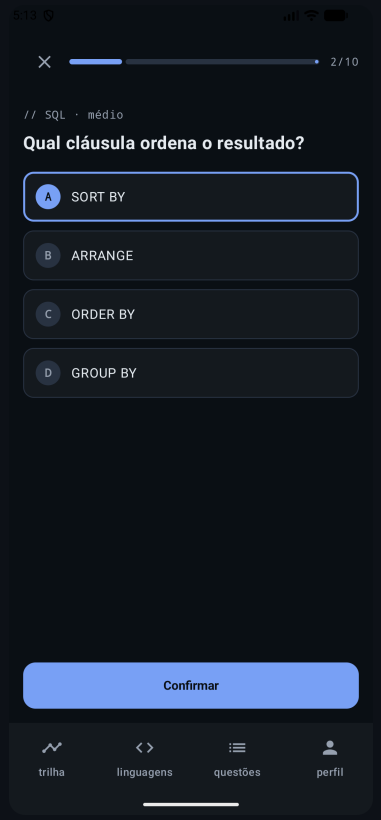
removida

// app rodando no emulador

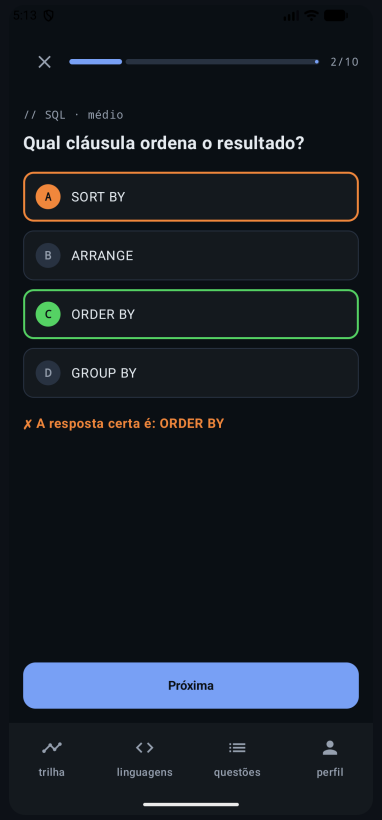
Fluxo de estudo



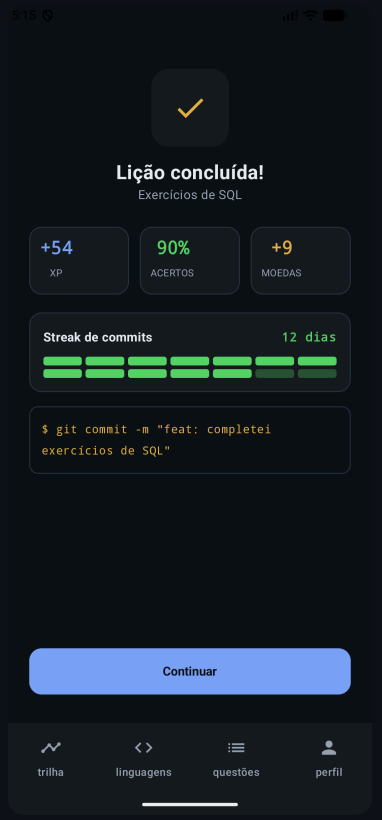
seleção



resposta escolhida

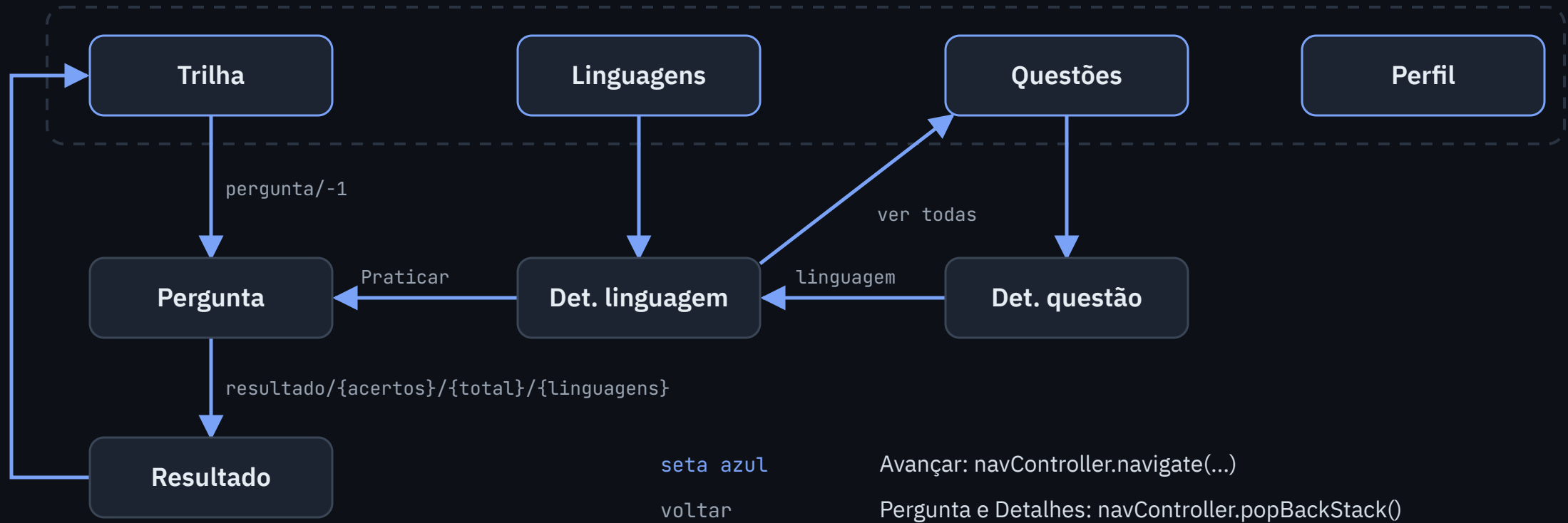


confirmada



90% · +54 XP

Fluxo de navegação



seta azul	Avançar: navController.navigate(...)
voltar	Pergunta e Detalhes: navController.popBackStack()
área tracejada	As 4 abas da NavigationBar

// arquitetura

As camadas do MVVM

View
ui/screens/

→

ViewModel
viewmodel/

→

Repository
data/repository/

→

Model
data/ · entidades, DAOs, Room

`leitura`
O Room emite um `Flow`. O ViewModel junta os dois com `combine`, copia para o `_uiState` e a tela observa com `collectAsStateWithLifecycle()`.

`escrita`
A tela chama um método do ViewModel, que valida e chama o Repository em `viewModelScope.launch`. Ninguém recarrega nada: o `Flow` emite sozinho.

// registro de decisões

Decisões do trio

O que decidimos, o que descartamos e por quê

Camadas

D1

Repository entre o ViewModel e o DAO

descartamos

A tela conhecer o DAO e saber que os dados vêm do Room

por quê

Se as questões vierem de uma API, só o Repository muda. ViewModel e telas continuam iguais

D2

DAO reativo, com Flow

descartamos

Chamar recarregarListas(...) depois de cada gravação

por quê

O Room emite a lista sozinho quando a tabela muda, e nenhuma tela precisa lembrar de recarregar

Estado

D3

UiState exposto por StateFlow

descartamos

Várias variáveis remember espalhadas em cada tela

por quê

_uiState é privado e o uiState é só leitura: só o ViewModel altera o estado

D4

Um ViewModel só para o quiz

descartamos

Deixar o progresso do quiz dentro da PerguntaScreen

por quê

O quiz tem estado próprio (perguntas, seleção, acertos) e recebe o linguagemId pela Factory

Organização

D5

Entidades em data e navegação em ui

descartamos

Manter as pastas model/ e navigation/

por quê

As entidades do Room são da camada de dados, e a navegação faz parte da interface

D6

Dados de exemplo no AppDatabase

descartamos

Inserir por um LaunchedEffect no AppNavigation

por quê

A navegação não conhece mais o banco. O onOpen só insere se o banco estiver vazio

// organização do código

Como dividimos os pacotes

com.example.duocode

```
MainActivity.kt
data/
├── Linguagem, Questao, Dificuldade
├── LinguagemDao, QuestaoDao
├── AppDatabase, Conversores
├── DadosIniciais
└── repository/
    ├── LinguagemRepository
    └── QuestaoRepository
viewmodel/
├── Linguagens  UiState + ViewModel + Factory
├── Questoes    UiState + ViewModel + Factory
└── Pratica     UiState + ViewModel + Factory
ui/
├── navigation/  AppNavigation, Rotas
├── screens/     as 8 telas
└── theme/
```

ui

Telas, navegação e tema. Só desenha e chama o ViewModel

viewmodel

Estado de cada tela e as ações do usuário

data.repository

A única porta de entrada para os DAOs

data

Entidades, DAOs e banco Room

DAO reativo e Repository

QuestaoDao.kt — data/QuestaoDao.kt

```
10 @Dao
11 interface QuestaoDao {
12     @Insert
13     suspend fun inserir(questao: Questao)
14
15     @Query("SELECT * FROM questoes")
16     fun buscarTodos(): Flow<List<Questao>>
—
22     suspend fun buscarPorLinguagem(linguagemId: Int):
   List<Questao>
23
24     @Update
25     suspend fun atualizar(questao: Questao)
26
27     @Delete
28     suspend fun deletar(questao: Questao)
29 }
```

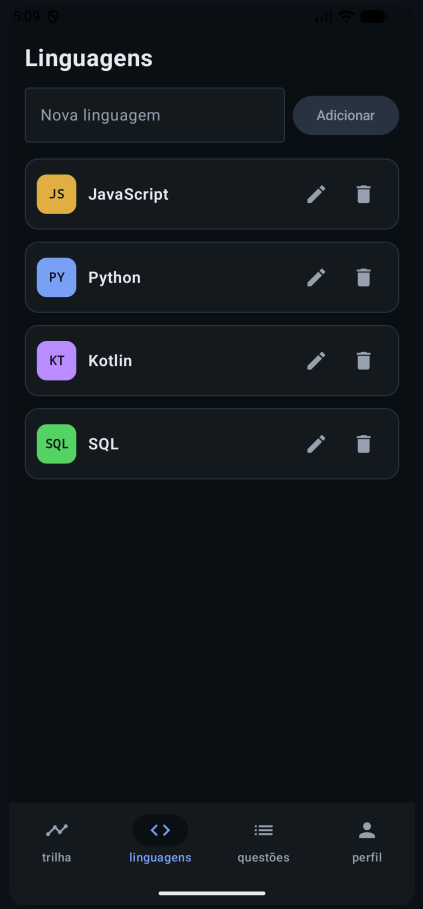
LinguagemRepository.kt — data/repository/LinguagemRepository.kt

```
 9 class LinguagemRepository(private val linguagemDao:
   LinguagemDao) {
10
11     fun getAllLinguagens(): Flow<List<Linguagem>> =
   linguagemDao.buscarTodos()
—
27 companion object {
28     fun criar(context: Context): LinguagemRepository =
29
   LinguagemRepository(AppDatabase.getDatabase(context).lin
   guagemDao())
30 }
```

ligação

buscarTodos() retorna Flow e não é suspend. O Repository só repassa as chamadas para o DAO, e o criar(context) evita que a tela cite o banco

ViewModel: Linguagens



LinguagensViewModel.kt — viewmodel/LinguagensViewModel.kt

```
22  private val _uiState = MutableStateFlow(LinguagensUiState())
23  val uiState: StateFlow<LinguagensUiState> = _uiState.asStateFlow()
—
27  combine(
28      linguagemRepository.getAllLinguagens(),
29      questaoRepository.getAllQuestoes()
30  ) { linguagens, questoes -> linguagens to questoes }
31      .catch { e -> Log.e("LinguagensViewModel", "Erro ao buscar as linguagens", e)
    }
32      .collect { (linguagens, questoes) ->
33          _uiState.update { it.copy(linguagens = linguagens, questoes = questoes) }
34      }
—
42  fun adicionarLinguagem() {
43      val state = _uiState.value
44      if (state.erroNovoNome != null) return
45
46      val nome = state.novoNome.trim()
47      val linguagem = Linguagem(nome = nome, sigla = nome.take(2).uppercase())
—
49  viewModelScope.launch {
50      try {
51          linguagemRepository.inserir(linguagem)
52          _uiState.update { it.copy(novoNome = "") }
```

estado
linhas 22–23

combine
linhas 27–30

validação
linha 44

inserir
linha 51

// estado das telas

O que tem dentro de cada UiState

LinguagensUiState

`linguagens, questoes`
dados do banco; questoes conta o que o CASCADE apaga

`novoNome`
campo "Nova linguagem"

`linguagemEmEdicao`
`null` = diálogo fechado

`nome/sigla/descricaoEdicao`
textos do diálogo

`linguagemParaRemover`
`null` = sem confirmação aberta

QuestoesUiState

`linguagens, questoes`
dados do banco; os Detalhes filtram pelo id

`carregando`
até o banco responder

`novaQuestao, edicao`
rascunhos dos formulários

`criandoQuestao`
diálogo do Detalhe da linguagem

`questaoEmEdicao`
`null` = diálogo fechado

PraticaUiState

`linguagemId`
`-1` = todas as linguagens

`linguagens, questoes`
dados do banco

`desmarcadas`
questões fora da prática

`perguntas`
`null` = ainda na seleção

`indiceAtual, selecionada, respondida, acertos`
progresso do quiz

Factory e tela

LinguagensViewModelFactory.kt — viewmodel/LinguagensViewModelFactory.kt

```kotlin

class LinguagensViewModelFactory(
    private val linguagemRepository: LinguagemRepository,
    private val questaoRepository: QuestaoRepository
) : ViewModelProvider.Factory {

    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        if (modelClass.isAssignableFrom(LinguagensViewModel::class.java)) {
            @Suppress("UNCHECKED_CAST")
            return LinguagensViewModel(linguagemRepository, questaoRepository) as T
        }
        throw IllegalArgumentException("Unknown ViewModel class")
    }
}
```

LinguagensScreen.kt — ui/screens/LinguagensScreen.kt

```kotlin
fun LinguagensScreen(
    navController: NavController,
    viewModel: LinguagensViewModel = viewModel(
        factory = LinguagensViewModelFactory(

LinguagemRepository.criar(LocalContext.current),
            QuestaoRepository.criar(LocalContext.current)
        )
    )
) {
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()
```

ligação

A Factory entrega os Repositories, e a tela só observa o uiState

ViewModel: Questões



QuestoesViewModel.kt — viewmodel/QuestoesViewModel.kt

```
53  fun adicionarQuestao() {
54      val state = _uiState.value
55      val linguagem = state.linguagemDaNovaQuestao ?: return
56      if (state.erroNovaQuestao != null) return
—
58  val formulario = state.novaQuestao
59  val questao = Questao(
60      enunciado = formulario.enunciado.trim(),
61      resposta = formulario.resposta.trim(),
62      linguagemId = linguagem.id,
63      dificuldade = formulario.dificuldade,
64      alternativasErradas = formulario.alternativasPreenchidas
65  )
—
67  viewModelScope.launch {
68      try {
69          questaoRepository.inserir(questao)
70          _uiState.update {
71              it.copy(
72                  novaQuestao = FormularioQuestao(linguagemId = linguagem.id,
    dificuldade = formulario.dificuldade),
73                  criandoQuestao = false
74              )
75          }
```

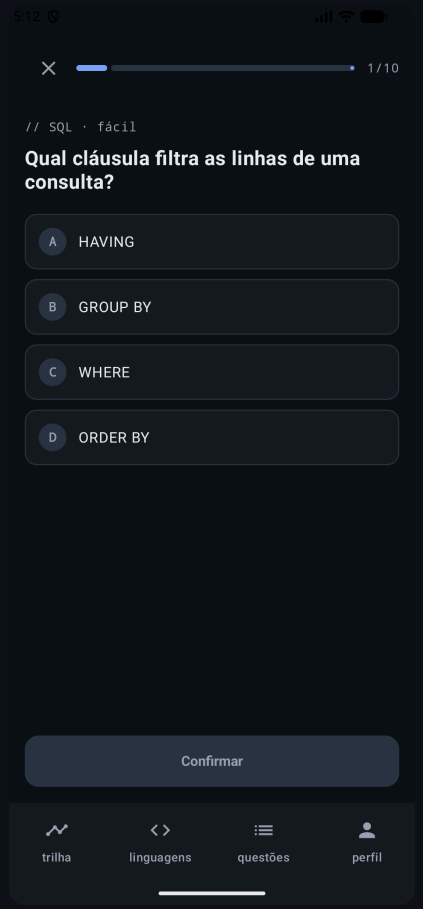
validação
linha 56

inserir
linha 69

limpa o formulário
linhas 72-73

regras
QuestoesUiState,
linhas 52-65

ViewModel: o quiz



PraticaViewModel.kt — viewmodel/PraticaViewModel.kt

```
71  _uiState.update { it.copy(respondida = true, acertos = if (state.acertou)
    it.acertos + 1 else it.acertos) }
—
73  if (state.acertou) {
74      viewModelScope.launch {
75          try {
76              questaoRepository.atualizar(pergunta.questao.copy(resolvida = true))
—
91  private fun montarPerguntas(selecionadas: List<Questao>, todas: List<Questao>):
    List<Pergunta> =
92      selecionadas.shuffled().map { questao ->
—
97      val erradas = (questao.alternativasErradas + reservas)
98          .map { it.trim() }
99          .filter { it.isNotBlank() && !it.equals(correta, ignoreCase = true) }
100         .distinctBy { it.lowercase() }
101         .take(QUANTIDADE_ALTERNATIVAS_ERRADAS)
102     val alternativas = (erradas + correta).shuffled()
103     Pergunta(questao, alternativas, alternativas.indexOf(correta))
104 }
```

acerto
linha 71

resolvida
linha 76

3 erradas
linha 101

embaralha
linha 102

// durante a reorganização

Alguma funcionalidade quebrou?

compilação

Pastas apagadas antes da hora

O que houve: o `220c455` apagou `model/` e `navigation/`, mas o `MainActivity` já importava `ui.navigation`.

Correção: o `c339c06` criou os arquivos em `data/` e `ui/navigation/`.

flow

Seed sem compilar

O que houve: um `Flow` não é lista, então `.isNotEmpty()` parou de funcionar no `DadosIniciais.kt`.

Correção: pegar a lista atual com `.first()`.

dados iniciais

Seed sem lugar

O que houve: o `AppNavigation` deixou de conhecer o banco, e o app abriria vazio.

Correção: inserir no `Callback.onOpen` do `AppDatabase`.

Conferido no emulador: adicionar, listar, editar, remover e persistir funcionam como no Trabalho 3.

O código sendo construído

quando	quem	commit	o que entrou	linhas
03/11 · 08:38	André Caires	220c455	Refatora para MVVM: ViewModels, Repositories e DAOs com Flow	+222 −568
03/11 · 11:43	Gustavo Cáceres	c339c06	Cria Repositories, ViewModels com UiState e Factories	+816 −226
03/11 · 11:44	Gustavo Cáceres	6b77a28	Merge do pull request #6	—
03/11 · 15:46	Gustavo Cáceres	bf91bfc	Cria o ViewModel do quiz com UiState e Factory	+236 −116
03/11 · 15:47	Gustavo Cáceres	ee80fe9	Merge do pull request #7	—

// dificuldades de cada integrante

Desafios e como resolvemos

André

220c455

desafio

Trocar o buscarTodos() para Flow fez o código que usava a lista parar de compilar.

como resolveu

Usou .first() para pegar a lista atual e levou o seed para o onOpen do AppDatabase.

Gustavo

c339c06 · bf91bfc

desafio

Tirar o estado das telas sem mudar o comportamento, principalmente as validações.

como resolveu

Deixou os erros como campos calculados no UiState e criou um PraticaViewModel para o quiz.

Brenno

conferência do checklist

desafio

As telas ainda pegavam o DAO para montar o Repository.

como resolveu

Criou um criar(context) em cada Repository e repetiu os testes no emulador.

Obrigado!

github.com/Gustavo-Caceres/Duo_Code_Parte_Tres

André Caires · Gustavo Cáceres · Brenno Soares